

A 30-DAY FIELD JOURNAL

The AI Trail

Thirty waypoints from “what even is AI?” to shipping it — one idea a day, in plain English.

- **Basecamp** Fundamentals Days 1–10
- **The Forge** Practical GenAI Days 11–20
- **Summit Ridge** ML Engineering Days 21–30

30 DAYS · 90 SLIDES · THE FULL COURSE

DAY 01 / 30

BASECAMP · FUNDAMENTALS

What is AI, really?

What is AI, really?

Everyone says "AI" like it's one thing. It isn't.

- ◆ Software that finds patterns in data instead of following hand-written rules
- ◆ A spam filter and a chatbot are both "AI" — just different points on a spectrum
- ◆ Ask what pattern it learned, and from what data, not "is this magic"

DAY 01 / 30

“If you could finally understand one thing about AI, what would it be? Drop it in the comments — I'll make sure it's covered somewhere in this series.”

Follow along — Day 2 drops tomorrow.

#AI

#MachineLearning

#LearnInPublic

#ArtificialIntelligence

DAY 02 / 30

BASECAMP · FUNDAMENTALS

AI, ML, Deep Learning, GenAI: the family tree

AI, ML, Deep Learning, GenAI: the family tree

I used to use these four terms interchangeably. That was a mistake — they're nested inside each other, like Russian dolls.

- ◆ Artificial Intelligence (AI) is the broadest term: any system that mimics intelligent behavior.
- ◆ Machine Learning (ML) is a subset of AI: systems that improve at a task by learning from data, rather than being explicitly programmed.
- ◆ Deep Learning is a subset of ML: it uses layered neural networks (more on those in a few days) to learn especially complex patterns — the technique behind image recognition and speech-to-text.
- ◆ Generative AI is a subset of deep learning: models trained to produce new content — text, images, audio — rather than just classify or predict a number.

DAY 02 / 30

“Which of these four terms do you think gets misused the most in your industry?”

Follow along – Day 3 drops tomorrow.

#AI

#MachineLearning

#DeepLearning

#GenerativeAI

DAY 03 / 30

BASECAMP · FUNDAMENTALS

How machines "learn": training data explained simply

How machines "learn": training data explained simply

The word "learn" makes it sound like the computer is thinking. It's really just adjusting numbers until it stops being wrong as often.

- ◆ The model guesses, gets corrected, adjusts its internal numbers — repeat billions of times
- ◆ No understanding required, just automated trial and error at massive scale
- ◆ Data quality sets the ceiling: garbage in, garbage out

“Next time you evaluate an AI product, ask what data it was trained on before you ask what algorithm it uses. The data question matters more.”

Follow along – Day 4 drops tomorrow.

#MachineLearning

#DataScience

#AI

#TrainingData

DAY 04 / 30

BASECAMP · FUNDAMENTALS

Supervised vs unsupervised vs reinforcement learning

Supervised vs unsupervised vs reinforcement learning

There are three main ways to teach a machine, and they map surprisingly well onto how humans learn too.

- ◆ Supervised learning: learning from labeled examples with a known right answer. Like flashcards — "here's the input, here's the correct output, learn the pattern." Used for spam detection, price prediction, medical image diagnosis.
- ◆ Unsupervised learning: finding structure in data with no right answers given. Like being handed a messy pile of photos and asked to sort them into groups without being told what the groups are. Used for customer segmentation, anomaly detection.
- ◆ Reinforcement learning: learning through trial, error, and reward — no labeled data at all, just a signal for "good" or "bad" outcomes. Like training a dog with treats. Used for game-playing AI, robotics, and — this part surprises people — a key step in training chatbots to be helpful (RLHF: [Reinforcement Learning from Human Feedback](#)).

DAY 04 / 30

“Which of these three feels most intuitive to you, and which still feels like a black box?”

Follow along – Day 5 drops tomorrow.

#MachineLearning

#ReinforcementLearning

#AI

#DataScience

DAY 05 / 30

BASECAMP · FUNDAMENTALS

Neural networks in plain English

Neural networks in plain English

Neural networks sound intimidating because of the name. The actual idea is simpler than most spreadsheets I've built.

- ◆ Layers of tiny weighted decision points passing signals forward
- ◆ Early layers learn simple patterns, later layers learn abstract ones
- ◆ "Neural" is a loose metaphor, not a brain simulation — it's weighted math

DAY 05 / 30

“Does knowing it's "just" weighted math make AI feel less magical, or more impressive?”

Follow along – Day 6 drops tomorrow.

#NeuralNetworks

#DeepLearning

#AI

#MachineLearning

DAY 06 / 30

BASECAMP • FUNDAMENTALS

What is a Large Language Model (LLM)?

What is a Large Language Model (LLM)?

An LLM — the tech behind ChatGPT, Claude, Gemini — is, at its core, a very sophisticated next-word predictor.

- ◆ At its core: a very sophisticated next-word predictor
- ◆ "Large" means huge training data plus billions of parameters
- ◆ No fact database — it generates plausible text, usually (not always) true

“Understanding "next-token prediction" is the single most useful mental model for using these tools well: you're not talking to a database, you're steering a very good pattern-completion engine.”

Follow along – Day 7 drops tomorrow.

#LLM

#GenerativeAI

#ChatGPT

#AI

DAY 07 / 30

BASECAMP • FUNDAMENTALS

Training vs inference: why ChatGPT doesn't "learn" from your chats

Training vs inference: why ChatGPT doesn't "learn" from your chats

A question I get a lot: "If I correct the AI in a conversation, does it remember that for next time?" Short answer: no. And understanding why clears up a lot of confusion about how these systems actually work.

- ◆ Training: the expensive, offline process (Day 3) where the model's weights get adjusted across a massive dataset. This happens once, over weeks, on huge clusters of specialized hardware, before you ever see the model.
- ◆ Inference: what happens every time you send a message. The model's weights are now frozen. It's just running the "arithmetic" from Day 5 forward — reading your input and generating output. No weights change during inference, ever.

DAY 07 / 30

“Did you assume AI chatbots were continuously learning from your conversations? You're not alone — most people do.”

Follow along – Day 8 drops tomorrow.

#AI

#LLM

#MachineLearning

#GenerativeAI

DAY 08 / 30

BASECAMP · FUNDAMENTALS

Tokens & embeddings: how AI understands language

Tokens & embeddings: how AI understands language

AI models don't read words. They read numbers. Here's how text becomes math.

- ◆ Tokenization chops text into word and sub-word chunks the model can process
- ◆ Embeddings turn tokens into numbers that capture meaning as geometry
- ◆ Similar meanings end up mathematically close together

DAY 08 / 30

“Once you see language as coordinates in space, a lot of "how does AI even understand me" stops feeling mysterious and starts feeling like math.”

Follow along – Day 9 drops tomorrow.

#AI

#NLP

#MachineLearning

#Embeddings

DAY 09 / 30

BASECAMP • FUNDAMENTALS

Hallucinations: why AI confidently makes things up

Hallucinations: why AI confidently makes things up

The single most important thing to understand before you trust AI output for anything important: it was never optimized to tell the truth. It was optimized to sound plausible.

- ◆ Asking for specific citations, statistics, or quotes
- ◆ Questions about very recent events (past the training cutoff)
- ◆ Obscure or niche topics with little training data
- ◆ Leading questions that assume a false premise

DAY 09 / 30

“What's the most confidently wrong answer AI has ever given you?”

Follow along – Day 10 drops tomorrow.

#AI

#Hallucination

#LLM

#ResponsibleAI

DAY 10 / 30

BASECAMP · FUNDAMENTALS

AI ethics & bias: the human side of the machine

AI ethics & bias: the human side of the machine

Ten days into this series, one theme keeps resurfacing: AI systems inherit everything from their data — including our flaws.

- ◆ Consent — was the data the model was trained on used with permission?
- ◆ Transparency — do users know they're interacting with AI, and how decisions get made?
- ◆ Accountability — when an AI system causes harm, who is responsible: the developer, the deployer, or the user?

DAY 10 / 30

“Which AI ethics issue concerns you most: bias, privacy, or accountability?”

Follow along – Day 11 drops tomorrow.

#AIEthics

#ResponsibleAI

#MachineLearning

#AI

DAY 11 / 30

THE FORGE · PRACTICAL GENAI

Your first great prompt: anatomy of a good instruction

Your first great prompt: anatomy of a good instruction

"Write me a marketing email" and "Write a 150-word marketing email for busy IT managers, announcing our new uptime SLA, in a confident but not salesy tone, with a single clear call to action" will get you two completely different quality levels of output. Same model. Same person. Wildly different results.

- ◆ Task — what exactly do you want done? Be specific about the verb: summarize, compare, rewrite, critique.
- ◆ Context — what does the model need to know that it can't guess? Your audience, your constraints, relevant background.
- ◆ Format — how should the output look? Length, structure, tone, bullet points vs prose.
- ◆ Examples (when it matters) — showing one sample of what "good" looks like often outperforms paragraphs of description. More on this tomorrow.

DAY 11 / 30

“What's the vaguest prompt that surprised you by working well anyway?”

Follow along – Day 12 drops tomorrow.

#PromptEngineering

#AI

#GenerativeAI

#Productivity

DAY 12 / 30

THE FORGE · PRACTICAL GENAI

Zero-shot vs few-shot prompting

Zero-shot vs few-shot prompting

One of the highest-leverage prompt engineering tricks costs you two extra sentences: showing an example.

- ◆ Zero-shot prompting: you ask for a task with no examples, relying purely on the model's general training. "Classify this review as positive or negative." Works fine for simple, common tasks.
- ◆ Few-shot prompting: you include a couple of example input/output pairs before your real request. Suddenly the model has a concrete pattern to match, not just an abstract instruction — and quality jumps, especially for tasks with a specific format, tone, or edge cases that are hard to describe in words.

DAY 12 / 30

“Have you tried few-shot prompting, or have you mostly been doing zero-shot without realizing it had a name?”

Follow along – Day 13 drops tomorrow.

#PromptEngineering

#AI

#LLM

#GenerativeAI

DAY 13 / 30

THE FORGE · PRACTICAL GENAI

Chain-of-thought: getting AI to "show its work"

Chain-of-thought: getting AI to "show its work"

Remember your math teacher insisting you show your work, not just the final answer? It turns out that trick makes AI models smarter too.

- ◆ Asking AI to "think step by step" gives it more reasoning room
- ◆ Writing out intermediate steps improves accuracy on logic and math
- ◆ Costs a few extra seconds of generation, often catches its own mistakes

DAY 13 / 30

“Try it today: take a prompt you'd normally send straight for an answer, and add "think step by step" before it. Notice the difference.”

Follow along – Day 14 drops tomorrow.

#PromptEngineering

#AI

#ChainOfThought

#LLM

DAY 14 / 30

THE FORGE · PRACTICAL GENAI

Role prompting: turning AI into an expert on demand

Role prompting: turning AI into an expert on demand

"You are a senior tax attorney specializing in small business deductions." One sentence, and the model's entire response shifts — vocabulary, structure, the questions it anticipates, even what it flags as a risk.

- ◆ Getting appropriately technical (or appropriately simple) language
- ◆ Surfacing the right caveats a domain expert would think to mention
- ◆ Adjusting tone — a "skeptical senior engineer" role catches different issues in a code review than a generic "review this code" prompt

DAY 14 / 30

“What's the most useful persona you've assigned an AI model?”

Follow along – Day 15 drops tomorrow.

#PromptEngineering

#AI

#GenerativeAI

#ProductivityTips

DAY 15 / 30

THE FORGE · PRACTICAL GENAI

Prompt iteration: treating prompts like code

Prompt iteration: treating prompts like code

The biggest mindset shift that improved my AI output quality: stop expecting the first prompt to be the final prompt.

- ◆ Write a first-draft prompt with your best guess at task, context, format (Day 11).
- ◆ Read the output critically — what's missing, what's wrong, what tone is off?
- ◆ Refine the prompt to address exactly that gap — don't start over, patch the specific issue.
- ◆ Repeat until the output is genuinely good, then save that prompt as a reusable template for next time.

DAY 15 / 30

*“Do you save and reuse
your best prompts, or rewrite
from scratch each time?”*

Follow along – Day 16 drops tomorrow.

#PromptEngineering

#AI

#GenerativeAI

#Productivity

DAY 16 / 30

THE FORGE · PRACTICAL GENAI

Retrieval-Augmented Generation (RAG) explained

Retrieval-Augmented Generation (RAG) explained

Remember Day 9: models hallucinate and their knowledge is frozen at a training cutoff. RAG is the most common fix, and it's more straightforward than the acronym suggests.

- ◆ Your question gets converted into an embedding (Day 8).
- ◆ The system searches a knowledge base — your company docs, a product manual, today's news — for chunks of text with the closest matching embeddings.
- ◆ Those retrieved chunks get inserted into the prompt as context.
- ◆ The model generates its answer grounded in that retrieved text, instead of purely from memory.

DAY 16 / 30

“Any tool you use daily is almost certainly RAG under the hood — customer support bots, internal doc search, "ask your codebase" tools.”

Follow along — Day 17 drops tomorrow.

#RAG

#AI

#LLM

#GenerativeAI

DAY 17 / 30

THE FORGE · PRACTICAL GENAI

AI agents: giving models tools and autonomy

AI agents: giving models tools and autonomy

A plain LLM can only do one thing: generate text. An "AI agent" is what you get when you wire that text-generation ability up to actions in the real world.

- ◆ A plain LLM only generates text; an agent can also take actions
- ◆ The loop: reason, pick a tool, get a result, decide the next step
- ◆ More autonomy means more power — and more surface area for mistakes

DAY 17 / 30

“Have you used an AI agent that surprised you with how much it could do on its own?”

Follow along – Day 18 drops tomorrow.

#AIAgents

#AI

#Automation

#GenerativeAI

DAY 18 / 30

THE FORGE · PRACTICAL GENAI

Multimodal AI: text, image, audio, and video together

Multimodal AI: text, image, audio, and video together

For years, AI models were specialists: one model for text, a different one for images, another for speech. Multimodal AI collapses that into a single model that can take in — and often generate — more than one type of data at once.

- ◆ Images and audio get chopped into patches and segments, then embedded like text
- ◆ One model can now reason across text, images, and audio together
- ◆ The frontier: real-time voice and video understanding

DAY 18 / 30

“What's the most useful non-text input you've fed an AI model — an image, a PDF, audio?”

Follow along — Day 19 drops tomorrow.

#MultimodalAI

#GenerativeAI

#AI

#ComputerVision

DAY 19 / 30

THE FORGE · PRACTICAL GENAI

Evaluating AI output: how do you know it's good?

Evaluating AI output: how do you know it's good?

"That looks right" is not an evaluation strategy — especially once you're using AI for anything repeated or high-stakes. Here's how to actually check your work.

- ◆ Factual claims — verify anything checkable (Day 9 territory) against a real source.
- ◆ Completeness — did it address every part of your prompt, or quietly drop a requirement?
- ◆ Fit for purpose — is the tone, length, and format actually usable as-is, or does it need editing?

DAY 19 / 30

“How do you personally sanity-check AI output before you trust it?”

Follow along – Day 20 drops tomorrow.

#AI

#Evaluation

#GenerativeAI

#ResponsibleAI

DAY 20 / 30

THE FORGE · PRACTICAL GENAI

Building your personal AI workflow toolkit

Building your personal AI workflow toolkit

Twenty days in, we've covered the "how it works" and the "how to prompt it." Today's post is about turning that into a system you actually use every day, instead of ad hoc chats you forget about.

- ◆ A prompt library — save your best prompts (Day 15) somewhere searchable, organized by task type: writing, analysis, coding, brainstorming.
- ◆ A default "context pack" — for recurring tasks, keep a short doc with the background the model always needs (your audience, your voice, your constraints) and paste it in rather than re-explaining every time.
- ◆ Task-fit awareness — know which tasks AI genuinely accelerates (first drafts, summarizing, brainstorming, boilerplate code) versus which ones it just feels fast on but need heavy verification (facts, final copy, anything with real consequences).
- ◆ A verification habit — pair every AI-assisted deliverable with the specific check from Day 19 that matches its stakes.

DAY 20 / 30

“What's one AI habit you've built that saves you real time every week?”

Follow along – Day 21 drops tomorrow.

#AI

#Productivity

#Workflow

#GenerativeAI

DAY 21 / 30

SUMMIT RIDGE · ML ENGINEERING

From notebook to product: what "ML engineering" means

From notebook to product: what "ML engineering" means

There's a huge gap between "I trained a model that works in my notebook" and "I shipped a model that works in production" — and that gap is basically the entire job of ML engineering.

- ◆ Handle messy, unexpected real-world input it never saw in training
- ◆ Respond fast enough that users don't notice or leave
- ◆ Serve thousands of requests at once instead of one
- ◆ Keep working when upstream data sources change or go down
- ◆ Be monitored so someone finds out when it starts failing, before customers do

DAY 21 / 30

“If you've ever built a model that worked great in testing and then struggled in the real world, what broke?”

Follow along – Day 22 drops tomorrow.

#MachineLearning

#MLEngineering

#AI

#DataScience

DAY 22 / 30

SUMMIT RIDGE · ML ENGINEERING

Data pipelines: the unglamorous backbone of AI

Data pipelines: the unglamorous backbone of AI

Nobody posts excitedly about data pipelines. And yet most ML teams will tell you they spend more time on data plumbing than on modeling itself.

- ◆ Extract, clean, transform, load — automated, and on a schedule
- ◆ No validation means bad data silently degrades your model over time
- ◆ Pipeline quality sets your entire AI system's quality ceiling

“The unglamorous truth: the quality ceiling of any AI system is set by its data pipeline, not its model architecture.”

Follow along – Day 23 drops tomorrow.

#DataEngineering

#MachineLearning

#AI

#MLOps

DAY 23 / 30

SUMMIT RIDGE · ML ENGINEERING

Feature engineering: making raw data model-ready

Feature engineering: making raw data model-ready

Raw data almost never comes in a form a model learns well from. Feature engineering is the craft of reshaping it so the patterns are easier for the model to find.

- ◆ Encoding categories as numbers — a model can't take "red/green/blue" directly; it needs a numeric representation.
- ◆ Scaling — putting features like "age" (0-100) and "income" (0-500,000) on comparable scales so one doesn't dominate purely due to magnitude.
- ◆ Creating derived features — "days since last purchase" is often far more predictive than raw purchase timestamps on their own; the model doesn't have to discover that transformation itself.
- ◆ Handling missing data thoughtfully — deleting, filling, or explicitly flagging gaps, depending on why the data is missing.

“Deep learning and LLMs shift some of this burden onto the model itself — but for structured business data, this skill is still core to the job.”

Follow along – Day 24 drops tomorrow.

#MachineLearning

#DataScience

#FeatureEngineering

#AI

DAY 24 / 30

SUMMIT RIDGE · ML ENGINEERING

Model evaluation metrics that actually matter

Model evaluation metrics that actually matter

"Our model is 95% accurate" sounds impressive until you learn the model is predicting fraud, and only 1% of transactions are actually fraudulent. A model that just guesses "not fraud" every single time also hits 95% accuracy — and catches zero fraud. This is why "accuracy" alone is one of the most misleading numbers in ML.

- ◆ Precision — of everything the model flagged as positive, how much was actually right? High precision means few false alarms.
- ◆ Recall — of everything that was actually positive, how much did the model catch? High recall means few missed cases.
- ◆ These trade off against each other — a model can catch every fraud case by flagging everything (perfect recall, terrible precision), so the right balance depends entirely on the cost of each type of error in your specific use case. Missing a fraud case and annoying a legitimate customer are not equally costly.

DAY 24 / 30

“What's a time a headline metric hid a real problem in your work?”

Follow along – Day 25 drops tomorrow.

#MachineLearning

#DataScience

#AI

#ModelEvaluation

DAY 25 / 30

SUMMIT RIDGE · ML ENGINEERING

Overfitting vs underfitting: the balancing act

Overfitting vs underfitting: the balancing act

The central tension in every model you'll ever train:
performing well on data it's already seen versus performing
well on data it hasn't.

- ◆ Overfitting: the model memorizes the training data — including its noise and quirks — instead of learning the underlying pattern. It scores great on training data and disappoints on anything new. Like a student who memorizes last year's exam answers instead of understanding the subject — perfect on that exact test, lost on a new one.
- ◆ Underfitting: the model is too simple to capture the real pattern at all, and performs poorly even on the training data. Like trying to fit a straight line to a curve — it misses the shape no matter how much data you give it.

“If a model's training score and real-world score tell wildly different stories, that gap itself is a diagnosis: it's telling you the model learned the wrong thing.”

Follow along – Day 26 drops tomorrow.

#MachineLearning

#DataScience

#AI

#ModelTraining

DAY 26 / 30

SUMMIT RIDGE · ML ENGINEERING

Fine-tuning vs prompting: when to customize a model

Fine-tuning vs prompting: when to customize a model

You want a model to behave a specific way for your use case. Do you write a better prompt, or retrain the model itself? This is one of the most common strategic decisions teams building on AI face — and prompting wins far more often than people assume.

- ◆ Prompting (including RAG from Day 16): you shape behavior entirely through the input, without touching the model's weights. Fast to iterate, cheap, easy to update, and surprisingly powerful for most tasks — including teaching a model new "knowledge" via retrieval.
- ◆ Fine-tuning: you continue training an existing model on your own examples, actually adjusting its weights (Day 3) so the new behavior becomes baked in. This is slower, more expensive, and requires real ML expertise — but it's the right call when you need a very specific output format at scale, a distinct tone/style baked in reliably, or behavior that's hard to reliably coax out through prompting alone.

“Skipping straight to fine-tuning when a better prompt would've solved it is one of the most common wastes of time and budget I see on AI projects.”

Follow along – Day 27 drops tomorrow.

#MachineLearning

#AI

#FineTuning

#GenerativeAI

DAY 27 / 30

SUMMIT RIDGE · ML ENGINEERING

Deploying models: APIs, latency, and scaling

Deploying models: APIs, latency, and scaling

A trained model, sitting on your laptop, helps nobody.

Deployment is the process of turning it into something other systems and users can actually call — and it introduces a whole new set of engineering constraints that have nothing to do with model accuracy.

- ◆ Serving via an API — the model gets wrapped behind an endpoint that accepts input and returns predictions, so any application can call it like any other web service.
- ◆ Latency — how fast is fast enough? A fraud check blocking a payment needs an answer in milliseconds. A report generated overnight can take minutes. The acceptable latency shapes which model size and hardware you can even consider.
- ◆ Scaling — one request is easy. Ten thousand simultaneous requests requires load balancing, request queuing, and often multiple model replicas running in parallel — plus a plan for what happens when demand spikes past capacity.
- ◆ Cost — every inference call (Day 7) consumes real compute. At scale, inference cost, not training cost, usually dominates the long-run budget of an AI product.

“The mindset shift from "building a model" to "deploying a model": accuracy gets you in the door, but latency, reliability, and cost determine whether the product actually survives contact with real users.”

Follow along – Day 28 drops tomorrow.

#MLOps

#MachineLearning

#AI

#SoftwareEngineering

DAY 28 / 30

SUMMIT RIDGE · ML ENGINEERING

MLOps: monitoring models after they ship

MLOps: monitoring models after they ship

Shipping a model isn't the finish line — it's the starting gun for a new problem: the real world keeps changing, and your model doesn't, unless you make it.

- ◆ Data drift — the incoming data starts looking statistically different from the training data (new customer behavior, a market shift, a seasonal change). The model is now making predictions on inputs it was never really trained for.
- ◆ Concept drift — the actual relationship between inputs and the right answer changes over time (what counted as "normal spending" before a recession isn't what counts as normal after one).

“Does your organization actively monitor deployed models for drift, or find out about problems from users first?”

Follow along – Day 29 drops tomorrow.

#MLOps

#MachineLearning

#AI

#DataScience

DAY 29 / 30

SUMMIT RIDGE · ML ENGINEERING

Cost & compute: the economics of running AI

Cost & compute: the economics of running AI

The most surprising thing to newcomers building their first AI product isn't a technical concept — it's the bill. Understanding where AI costs actually come from changes how you design a product from day one.

- ◆ Training cost — a one-time (or periodic) expense, often enormous for large models, but amortized across every future use.
- ◆ Inference cost — what you pay every single time the model runs (Day 7). For a product with real usage, this dwarfs training cost over time — it's the ongoing "cost of goods sold" for an AI feature, not a one-off R&D expense.
- ◆ The levers that actually move the number: model size (bigger models cost more per call), output length (you're often billed per token generated), prompt length (more context in = more cost), and call frequency (does every user action trigger a model call, or only the ones that need one?).

“The teams that treat inference cost as a design constraint from the start build meaningfully more sustainable AI products than the ones who bolt on efficiency after launch.”

Follow along – Day 30 drops tomorrow.

#AI

#MLOps

#Cost

#GenerativeAI

DAY 30 / 30

SUMMIT RIDGE · ML ENGINEERING

Putting it all together: your *AI* learning roadmap

Putting it all together: your AI learning roadmap

Thirty days ago, "AI" was a vague, slightly intimidating buzzword. If you've followed along, you now have a real mental model of how it works end to end:

- ◆ Phase 1 — the fundamentals: what AI/ML/deep learning/GenAI actually mean, how machines learn from data, how neural networks and LLMs work, why hallucinations happen, and the ethical stakes underneath it all.
- ◆ Phase 2 — using it well: how to write prompts that actually get good results, chain-of-thought, role prompting, RAG, agents, multimodal models, and how to evaluate whether AI output is genuinely good.
- ◆ Phase 3 — how it gets built and shipped: data pipelines, feature engineering, evaluation metrics, overfitting, fine-tuning vs prompting, deployment, monitoring, and the real economics of running AI in production.

DAY 30 / 30

*“Thank you for following this series.
Which day's concept are you going
to actually put into practice first?”*

Series complete – thanks for following the trail.

#AI

#MachineLearning

#GenerativeAI

#LearnInPublic